

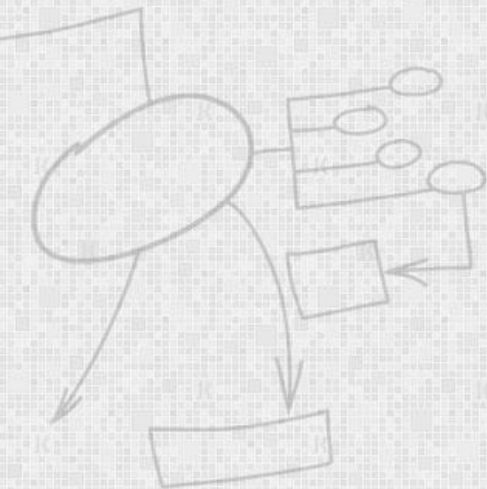
EC-360

SOFTWARE ENGINEERING

Lecture 13

Maintenance

By
Dr Wasi Haider Butt



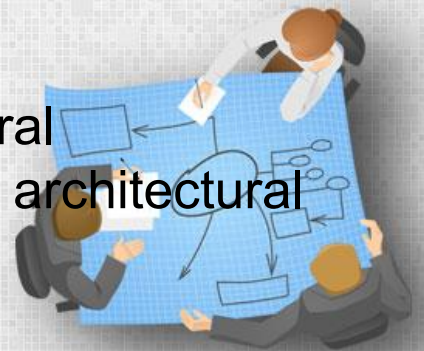
Software Evolution

- It is impossible to produce system of any size which do not need to be changed.
- Once software is **put into use**, new requirements emerge and existing requirements changes as the business running that software **changes**.
- Parts of the software may have to be modified to correct **errors** that are found in operation, improve its performance or other non-functional characteristics.
- All of this means that, after delivery, software systems always evolve in response to demand for change.



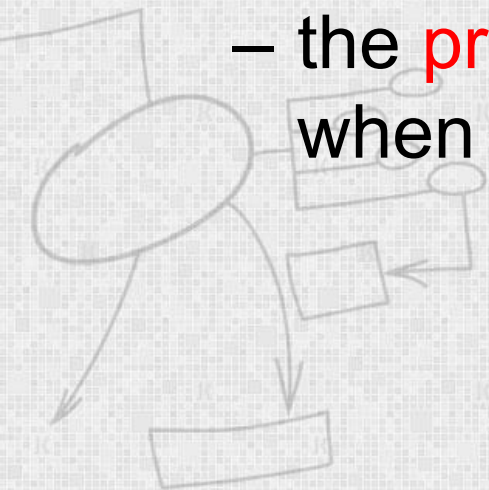
Software Evolution Approaches

- Software maintenance
 - Changes to the software are made in response to changed requirements but the **fundamental structure** of the **software remains stable**. This is most common approach used to system change.
- Architectural transformation
 - This is a more **fundamental** approach to software change than maintenance as it involves making significant change to the architecture of the system.
- Software re-engineering
 - System re-engineering may involve some structural modifications but does not usually involves major architectural change.



Maintenance Characteristics

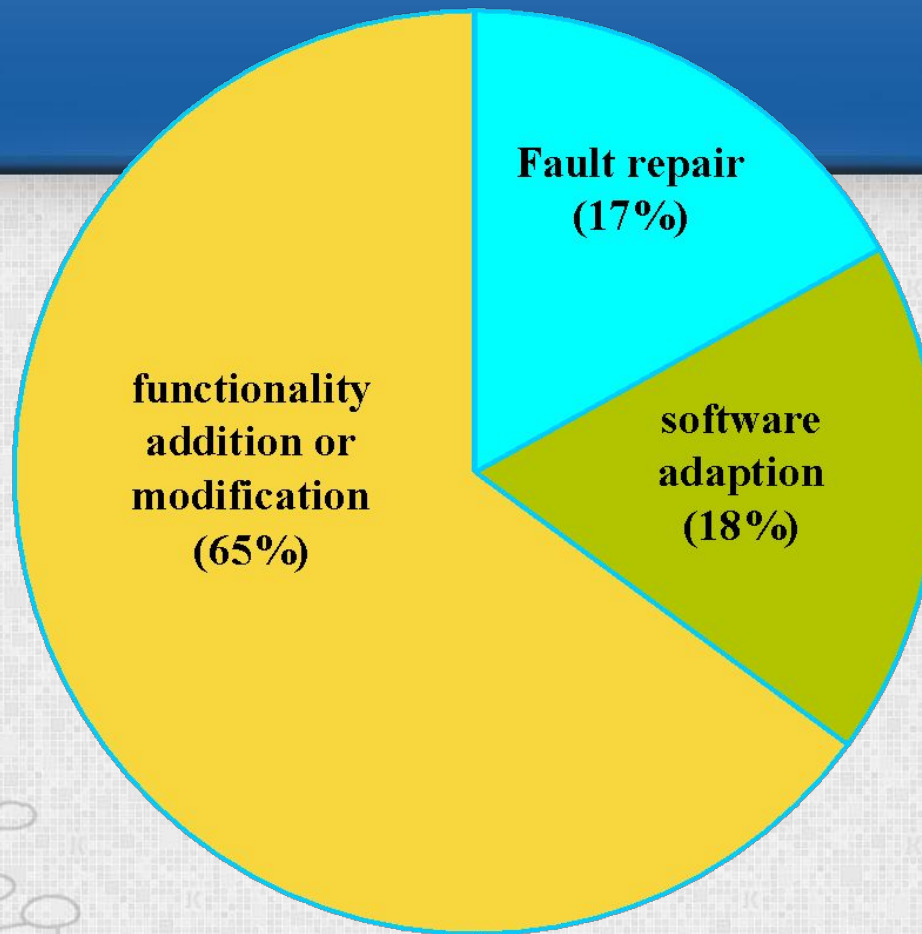
- We need to look at maintenance from three different viewpoints:
 - the **activities** involved
 - the **costs** associated with the maintenance phase
 - the **problems** that are frequently encountered when software maintenance is undertaken



Types of Maintenance

- Maintenance to repair software faults
 - Changing a system to correct deficiencies in the way meets its requirements
- Maintenance to adapt software to a different operating environment
 - Changing a system so that it operates in a different environment (computer, OS, etc.) from its initial implementation
- Maintenance to add to or modify the system's functionality
 - Modifying the system to satisfy new requirements



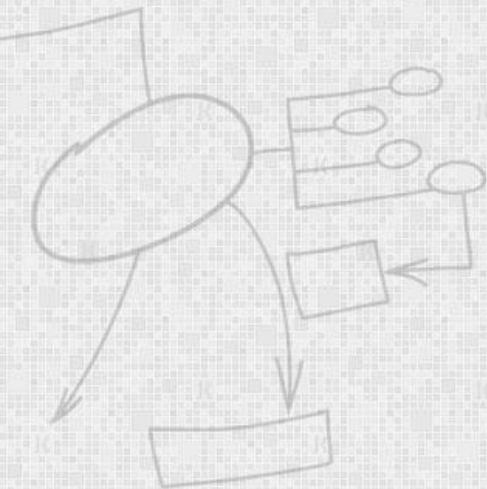


Maintenance effort distribution .[SOM2004]



Maintenance Examples

- Y2K
 - many, many systems had to be updated
- Anti-Virus Software
 - don't usually have to update software, but must send virus definitions



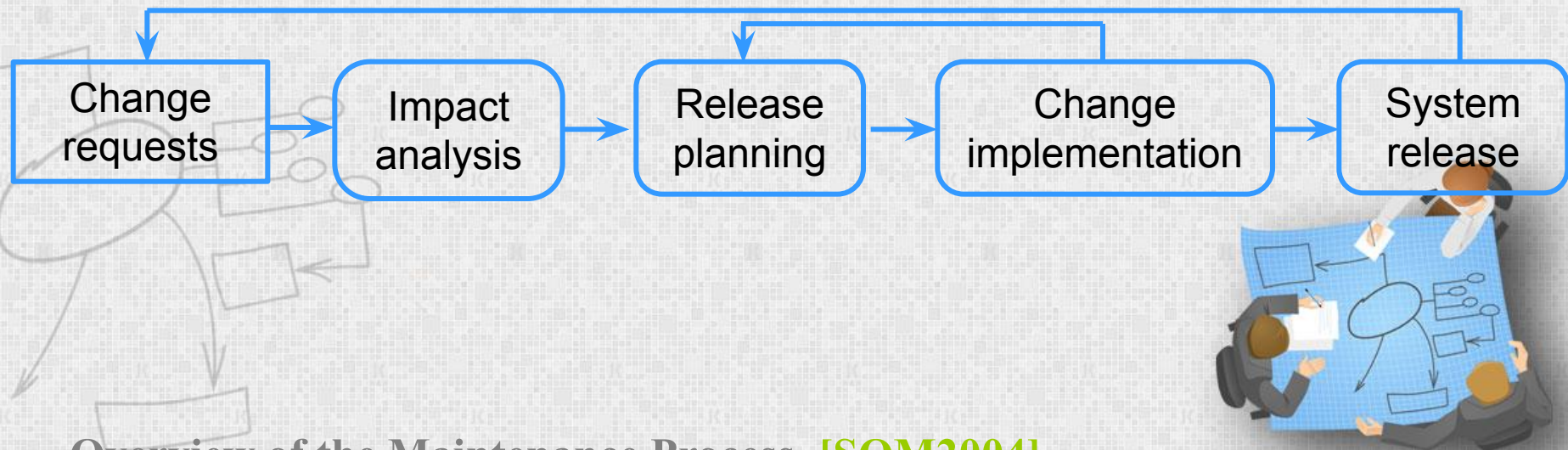
Maintenance Examples (cont'd)

- Operating System Patching
 - Microsoft, Apple, Linux/Unix
 - OS is core to use of computer, so it must be constantly maintained
- Commercial Software in General
 - customers need to be informed of updates
 - updates have to be easily available - web is good tool



The Maintenance Process

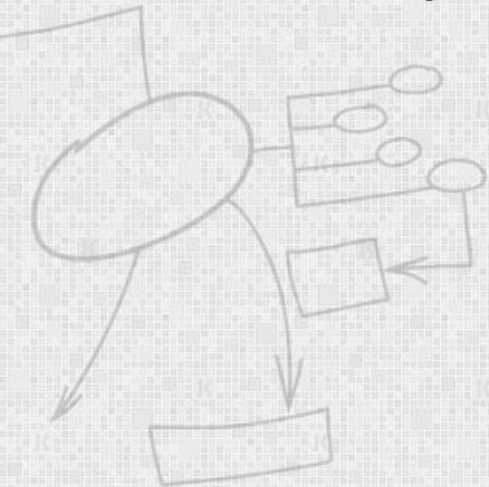
- Maintenance process vary considerably depending on the **types of software** being maintained, the **development processes** used in an organization and **people** involved in the process.



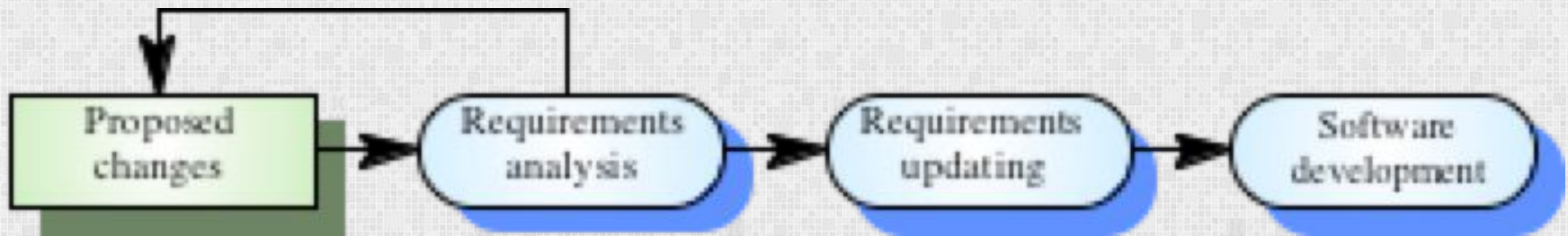
Overview of the Maintenance Process [\[SOM2004\]](#)

Change Requests

- Requests for **system changes** from stakeholders
- In practice, some change requests must be implemented **urgently**
 - Fault repair
 - Changes to the system's environment
 - Urgently required business changes



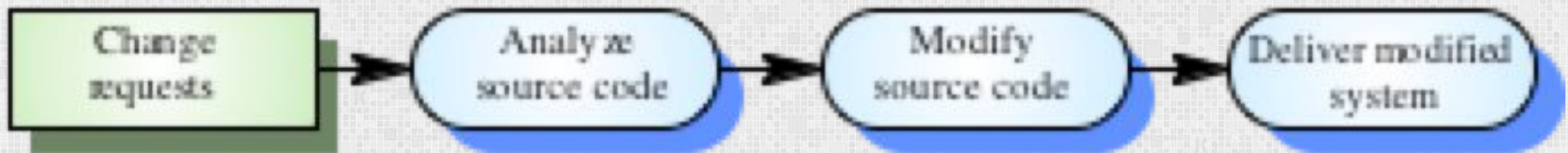
Change Implementation



Change implementation. [\[SOM2004\]](#)



Emergency Repair

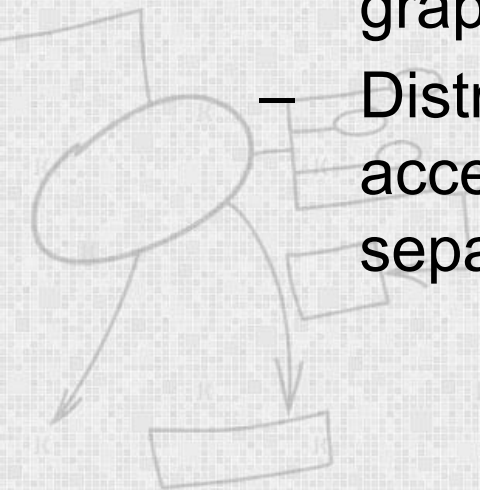


Emergency repair [SOM2004]



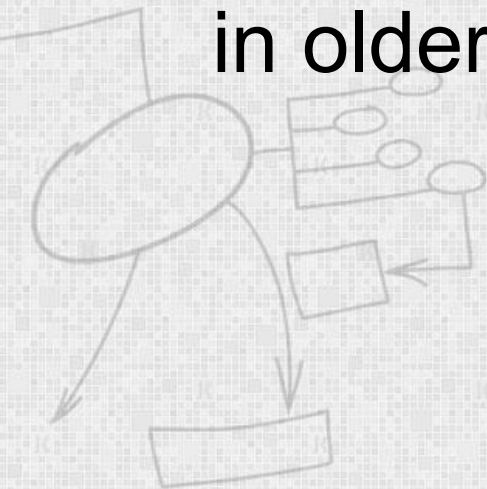
Architectural Evolution

- There is a need to convert many legacy systems from a centralised architecture to a client-server architecture
- Change drivers
 - User interface expectations. Users expect graphical user interfaces
 - Distributed access to systems. Users wish to access the system from different, geographically separated, computers

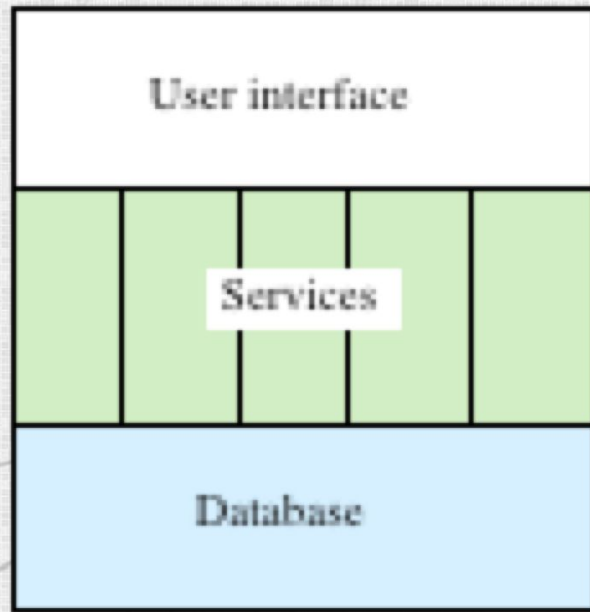


Legacy System Structure

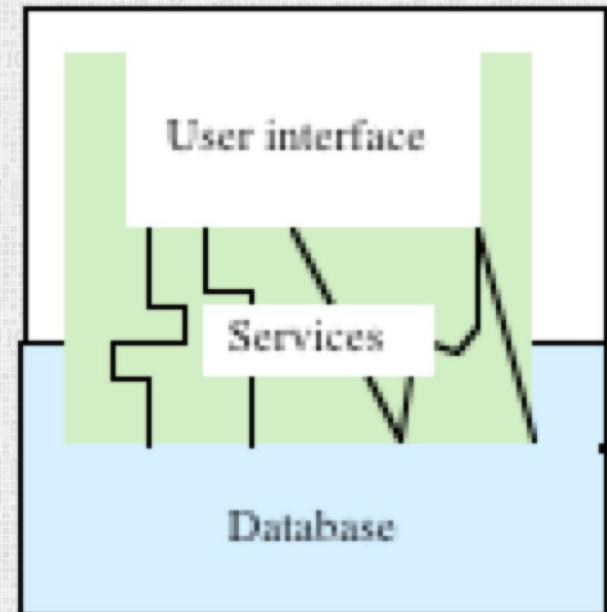
- Ideally, for distribution, there should be a clear separation between the user interface, the system services and the system data management
- In practice, these are usually **intermingled** in older legacy systems



Legacy System Structures



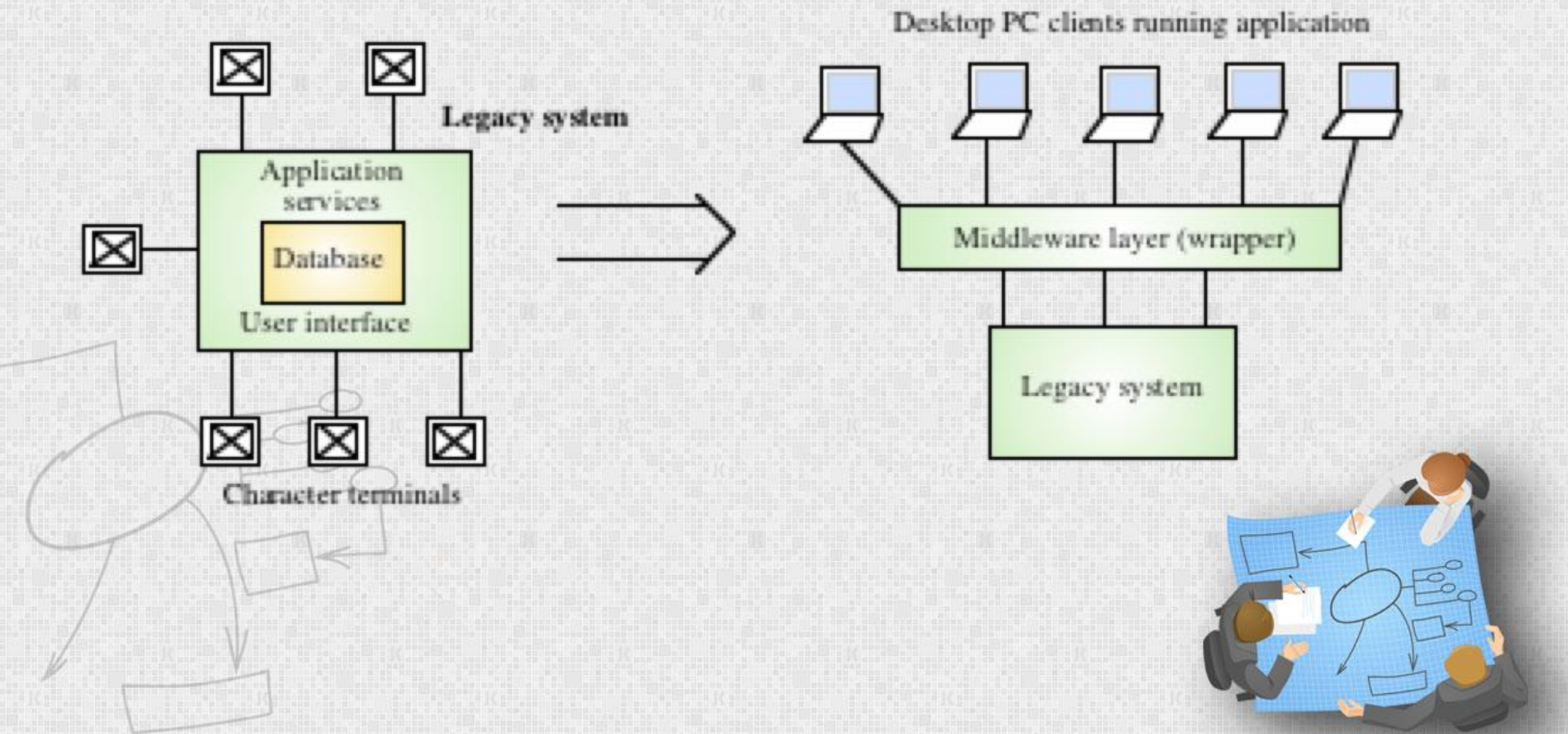
Ideal model for distribution



Real legacy systems

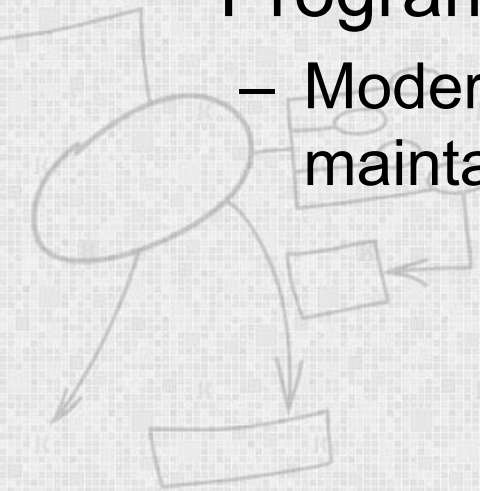


Legacy System Distribution



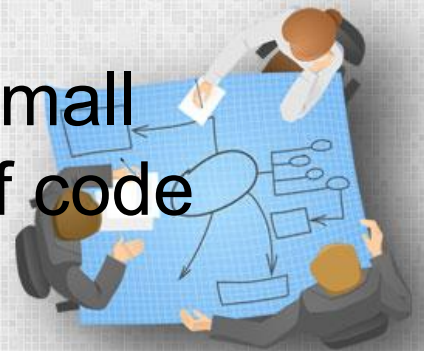
What Affects the Maintainability of an Application?

- Application age
 - (software rust?) older programs were probably worse written and have probably been patched more
- Size
 - measured in KLOC, number of input/output files
- Programming language
 - Modern tools are supposed to produce more maintainable code than Older



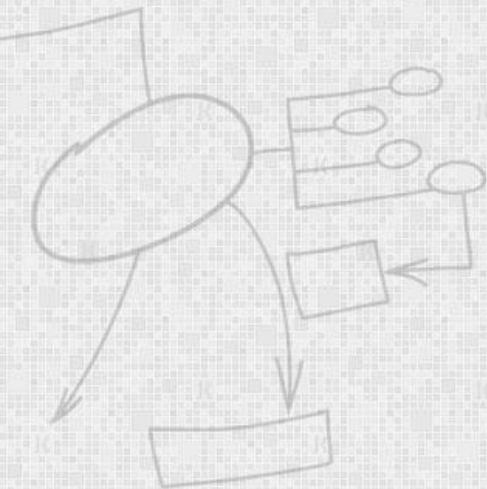
(cont'd)

- Processing environment
 - files harder to maintain than databases, real-time harder than batch
- Analysis and design methodologies
 - well designed software is supposed to be much easier to maintain
- Modularization
 - (central of all the OO techniques) small reasonably self contained pieces of code should be easier to maintain



(cont'd)

- End-user involvement
 - some researchers believe when end users are more involved maintenance decreases



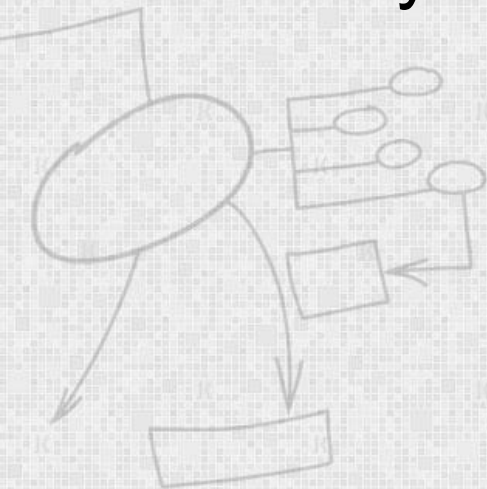
Maintenance Costs

- Usually **greater** than development costs
- Affected by both technical and non-technical factors
- Increases as software is maintained.
Maintenance corrupts the software structure so makes further maintenance more difficult.
- Ageing software can have high support costs (e.g. old languages, compilers etc.)



Change Management

- Change management process ensure that changes are **recorded** and **applied** to the system in a **cost-effective way**.
- Designed to ensure that the **costs** and **benefits** of change are properly **analyzed** and changes to a system are made in a **controlled way**.



Change Management

- Software change isn't a bad thing.
- But change always has a price.
- Unless project stakeholders manage changes during development, they won't really **know what will be delivered**.
- **Uncontrolled change**
 - project chaos
 - schedule slips
 - quality problems



Change Management

- **Proposed** requirement changes are carefully **evaluated** before being committed to.
- The **appropriate individuals** make **informed business** decisions about requested changes.
- Approved changes are **communicated** to all affected participants.
- The project incorporates requirements changes in a **consistent** fashion.



Originator submitted
a change request

Submitted

Evaluator performed
impact analysis

Evaluated

CCB decided
not to make
the change

Rejected

CCB decided to make the
change, allocated it to a release,
and assigned a Modifier

Approved

Change was
canceled; back
out of modifications

Modifier has made the
change and requested
verification

Verification failed

Change Made

Change was
canceled; back
out of modifications

Canceled

Verifier has
confirmed the
change

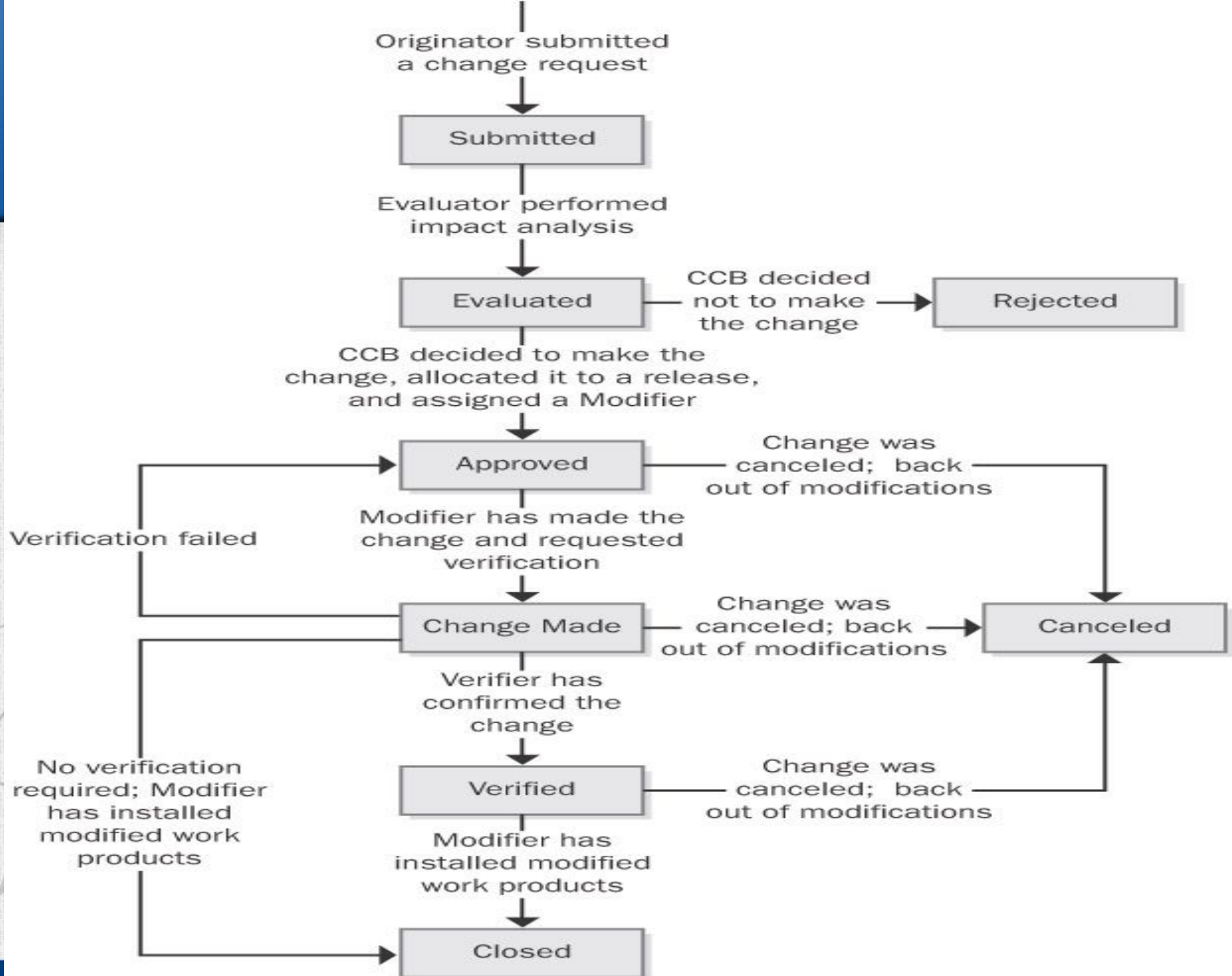
Verified

Change was
canceled; back
out of modifications

No verification
required; Modifier
has installed
modified work
products

Modifier has
installed modified
work products

Closed



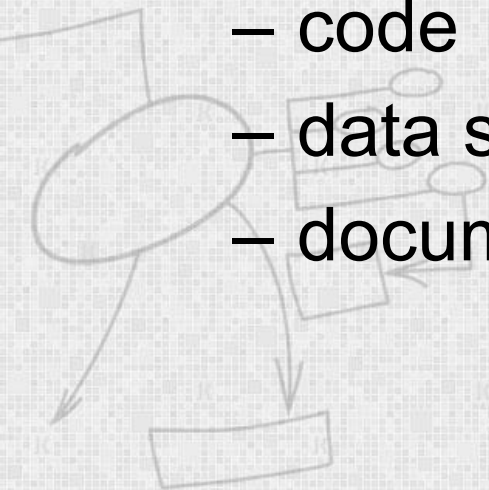
Impact Analysis

- For Decision Support about Acceptance/Rejection
- Identify components that might have to be created, modified, or discarded and to estimate the effort associated with implementing the change.
- Ripple Effect



Maintenance Side Effects

- In this context a side effect implies an error or undesirable behavior that occurs as the result of a modification.
- the three major areas are
 - code
 - data structures
 - documentation



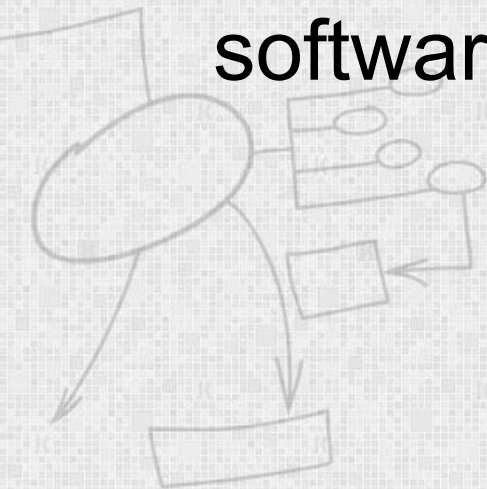
Documentation Side Effects

- These consist of the failure to update documentation so that it no longer matches the code.
- The entire documentation should be reviewed before re-release



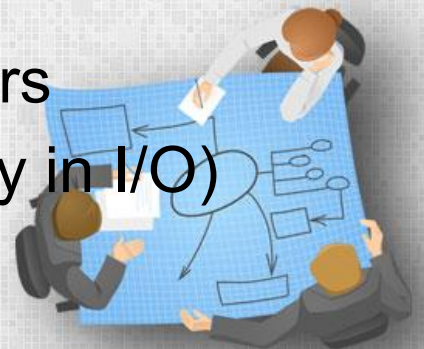
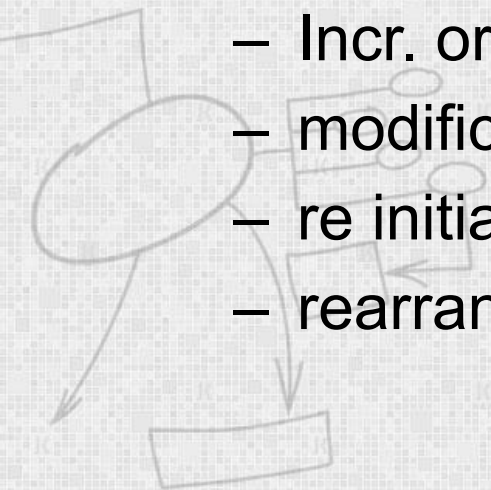
Coding Side Effects

- A subprogram is deleted or changed
- An identifier is deleted or modified
- Logical operators are modified
- These may be caught in testing or cause software failure during operation.



Data Side Effects

- Data side effects occur as the result of modifications made to a data structure. The most error-prone are:
 - redefinition of local and global constants
 - redefinition of record or file formats
 - Incr. or decr. in size of array or other data structure
 - modification of global data
 - re initialization of control flags and pointers
 - rearrangements of parameters (especially in I/O)



Program structure improvement

- Maintenance tends to corrupt the structure of a program. It becomes harder and harder to understand
- The program may be automatically restructured to remove unconditional branches
- Conditions may be simplified to make them more readable



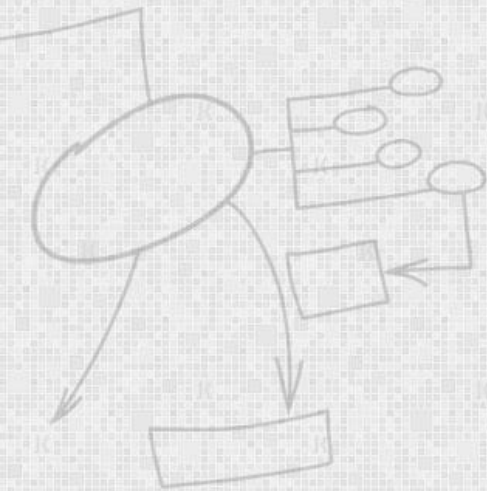
Condition simplification

-- Complex condition

if not (A > B and (C < D or not (E > F)))...

-- Simplified condition

if (A <= B or (C>= D and E > F)...



References

- [PRE2004] Roger S. Pressman. Software Engineering: a practitioner's approach, 6th edition. McGRAW-HILL, 2004.
- [SOM2004] Ian Sommerville. Software Engineering, 7th edition. Addison Wesley, 2004

